Name : JETHAVA JAY VINODBHAI

Enrollment No : 250283152004

1.Import Numpy

```
import numpy as np
```

2.Numpy Array: Array with one dimention -> Vectors, Two dimentions -> Matrices, Three or more dimentions -> Tensors

a. Create numpy array and print it. Use np.array

b. Create a two-dimentional array and print it.

c. Use the shape function to determine the shape of array.

a. 1D Array

```
arr= np.array([10,20,30,40,50])
print(arr)
```

```
[10 20 30 40 50]
```

b. 2D Array

```
arr2= np.array([[10,20,30],
 [40,50,60]])
print(arr2)
```

```
[[10 20 30]
 [40 50 60]]
```

c. Shape of Array

```
print(arr.shape)
print(arr2.shape)
```

```
(5,)
(2, 3)
```

d. Use reshape function to convert vector into matrix and matrix into vector.

```
vector=np.array([1,2,3,4,5,6])

matrix=vector.reshape(2,3)

print(matrix)

vector_again=matrix.reshape (6)

print(vector_again)
```

```
[[1 2 3]
 [4 5 6]]
[1 2 3 4 5 6]
```

3.Creating Arrays

a. Crate array using numpy arange function

```
a=np.arange(12)
print(a)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

b.Reshape it into a matrix form using reshape function.

```
matrix = a.reshape(3,4)

print(matrix)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

c. Create arrays filled with zeros and ones. Use zeros and ones function.

```
zeros=np.zeros((3,3))

ones=np.ones((4,3))

print("Zeros")

print(zeros)

print("Ones")

print(ones)
```

```
Zeros
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
Ones
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

d. Create two arrays with zeros and ones and combine them using:

```
zeros_compatible =np.zeros((3,3))
ones_compatible = np.ones((3,3))

print("Original zeros shape:", zeros.shape)

print("Original ones shape:", ones.shape)

print("combine Tow Array: \n")

print(np.column_stack((zeros_compatible, ones_compatible)))
```

```
Original zeros shape: (3, 3)
Original ones shape: (4, 3)
combine Tow Array:

[[0. 0. 0. 1. 1. 1.]
```

```
 [0. 0. 0. 1. 1. 1.]
 [0. 0. 0. 1. 1. 1.]]
```

```
#11. as rows underneath each other using row_stack

print(np.row_stack((zeros, ones)))
```

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
/tmp/ipykernel_2027/244335145.py:3: DeprecationWarning: `row_stack` alias is deprecated. Use `np.v
  print(np.row_stack((zeros, ones)))
```

4.Vectorised Functions:

a. Create a matrix of size 3X4 using arrange.

```
arr = np.arange(12)

arr = arr.reshape(3,4)

print(arr)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

b. Add 150 to each element

```
print(arr+150)
```

```
[[150 151 152 153]
 [154 155 156 157]
 [158 159 160 161]]
```

d. Create the array a ## ([[2, 4, 6, 8, 10], [12, 14, 16, 18, 20], [22, 24, 26, 28, 30], [32, 34, 36, 38, 40]])

```
a=np.arange(20).reshape(4,5)

a=(a+1)*2

print(a)
```

```
[[ 2  4  6  8 10]
 [12 14 16 18 20]
 [22 24 26 28 30]
 [32 34 36 38 40]]
```

e.Use comparison operators(a>6,a==10)

```
print(a>6)
print(a==10)
```

```
[[False False False  True  True]
 [ True  True  True  True  True]
 [ True  True  True  True  True]
 [ True  True  True  True  True]]
[[False False False False  True]
```

```
 [False False False False False]
 [False False False False False]
 [False False False False False]]
```

f. Use logical operators. ((a <3) | (a > 8)), ((a > 4) & (a < 7)), (~(a > 6))

```
print((a< 3) | (a > 8), "\n")

print((a > 4) & (a < 7),"\n")

print(~(a > 6), "\n")
```

```
[[ True False False False  True]
 [ True  True  True  True  True]
 [ True  True  True  True  True]
 [ True  True  True  True  True]]

[[False False  True False False]
 [False False False False False]
 [False False False False False]
 [False False False False False]]

[[ True  True  True False False]
 [False False False False False]
 [False False False False False]
 [False False False False False]]
```

5.Array Indexing and Slicing - Position: Indexing refer to extracting elements based on their position or certain criteria. This is one of the fundamental operations with arrays.

a. Create an array with 12 elements.

```
a=np.arange(12)

print(a)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

b. Make all elements from index 7 to 11 to -1.

```
a[7:12] = -1

print(a)
```

```
[ 0  1  2  3  4  5  6 -1 -1 -1 -1 -1]
```

c. Extract elements at index 2,3,6 positions in one go.

```
print(a[[2,3,6]])
```

```
[2 3 6]
```

d. Create a matrix of size 3X4.

```
m = np.arange(12).reshape(3,4)

print(m)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

e. Extract 2nd row

```
print(m[1])
```

```
[4 5 6 7]
```

f. Extract 3rd column

```
print(m[2])
```

```
[ 8  9 10 11]
```

g. Extract 1st, 2nd row

```
print(m[0:2])
```

```
[[0 1 2 3]
 [4 5 6 7]]
```

h. Extract 2nd row, 3rd column

```
print(m[1:2])
```

```
[[4 5 6 7]]
```

i. Extract 1s, 2nd row, first three columns

```
print(m[0:2,0:3])
```

```
[[0 1 2]
 [4 5 6]]
```

j. set the third row to 1,2,3,4

```
m[2] = [1,2,3,4]

print(m)
```

```
[[0 1 2 3]
 [4 5 6 7]
 [1 2 3 4]]
```

Double-click (or enter) to edit

6.Array Indexing and Slicing - Criteria

a. Create two vectors:

```
names = np.array(["Rahul", "Simran", "Ritesh", "Simran", "Rahul"])
score = np.array ([126, 115, 130, 141,132])
```

b. Extract all test scores that are smaller than 130

```
print(score[score < 130])
```

```
[126 115]
```

c. Scores of Simran

```
print(score[names == "Simran"])
```

```
[115 141]
```

d. Add 10 to Rahul's scores

```
score[names == "Rahul"] += 10

print(score)
```

```
[136 115 130 141 142]
```

7.Random numbers

a. random.choice()

```
print(np.random.choice ([10,20,30,40], size=5))
```

```
[20 20 40 20 40]
```

b. random.normal()

```
print(np.random.normal(0,1,5))
```

```
[-0.04117737 -1.02662677  0.63231556  0.806729   -0.75035137]
```

c. random.binomial()

```
print(np.random.binomial(10,0.5,5))
```

```
[5 4 5 2 5]
```

d. random.uniform()

```
print(np.random.uniform (1,10,5))
```

```
[9.77066888 6.73951631 7.39357258 9.84357465 3.89032381]
```

e. random.seed(

```
np.random.seed (42)
print(np.random.randint(1,100,5))
```

```
[52 93 15 72 61]
```

8.Statistical Functions

a. Create matrix

```
a=np.arange(1,13).reshape(3,4)

print(a)
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

b. Sum of all elements

```
print(np.sum(a))
```

```
78
```

c. Add rows, preserve columns

```
print(np.sum(a, axis=0))
```

```
[15 18 21 24]
```

d. Add columns, preserve rows

```
print(np.sum(a, axis=1))
```

```
[10 26 42]
```

Other Statistical Functions

```
print("Mean:", np.mean(a))

print("Median:", np.median(a))

print("Maximum", np.max(a))

print("Minimum", np.min(a))

print("Standard Deviation:", np.std(a))

print("Variance", np.var(a))
```

```
Mean: 6.5
Median: 6.5
Maximum 12
Minimum 1
Standard Deviation: 3.452052529534663
Variance 11.916666666666666
```